¿Bastan los parches?

Análisis crítico y réplica parcial de *Patches Are All You Need?* (Trockman y Kolter, TMLR 2023)

Marcos David Kantor

Docentes: Dr. Ing. Martín Morales y Dr. Ing. Marcelo Cappelletti

Universidad Nacional Arturo Jauretche — Doctorado en Ciencia y Tecnología

Curso de posgrado: Tópicos en Aprendizaje Automático — Trabajo Final Integrador

Diciembre de 2026



Contenido de la exposición

1. **El artículo** y por qué cumple la consigna.
2. **Introducción**: el problema y la motivación.
3. **Explicación teórica** del ConvMixer.
4. **Metodología** de los autores.
5. **Resultados** y **conclusiones** del artículo.
6. **Réplica parcial** en CIFAR-10, comparada con modelos de las cinco unidades.
7. **Reflexión crítica**.

Exposición de 20 minutos + 10 de debate.

En una frase

Una red convolucional de seis líneas, que recibe la imagen en *parches* como el ViT, compite con los transformers: parte del mérito estaría en los parches y no en la atención.

82,7 %

ConvMixer-128/8 en CIFAR-10, réplica en CPU, 10 épocas

(paper: 95,0 % con 256/8 y 200 épocas en GPU)

El artículo seleccionado

Trockman, A. y Kolter, J. Z. (Carnegie Mellon / Bosch Center for AI)
Patches Are All You Need?
Transactions on Machine Learning Research, 2023 — arXiv:2201.09792 (2022)
Código: `github.com/locuslab/convmixer`

Cumple la consigna

- Reciente (2022–2023) y con un problema relevante: clasificación de imágenes.
- Propone un modelo y lo compara con ResNet, ViT, DeiT, MLP-Mixer, ResMLP y Swin.
- Resultados en ImageNet-1k y CIFAR-10, este último **público y replicable**.

Ubicación en el programa

U4 (CNN): convoluciones 2D, SGD, arquitecturas y modelos.

La réplica agrega un modelo de cada unidad: **U1** validación cruzada, **U2** seudoinversa y perceptrón, **U3** MLP, **U5** ELM y ML-ELM.

Es bibliografía de la materia *Sistemas de Inteligencia Artificial* (ITBA) que cursé en paralelo.

El problema: el ViT cambió dos cosas a la vez

CNN clásica (ResNet; He et al., 2016)

Ve píxeles. Es **piramidal**: la resolución baja y los canales suben capa a capa. Su sesgo inductivo es la localidad y los pesos compartidos.

Vision Transformer (2021)

(Dosovitskiy et al., 2021). Corta la imagen en **parches** de 16×16 (la atención cuesta $O(n^2)$). Es **isotrópico**: igual resolución y ancho en todas las capas.

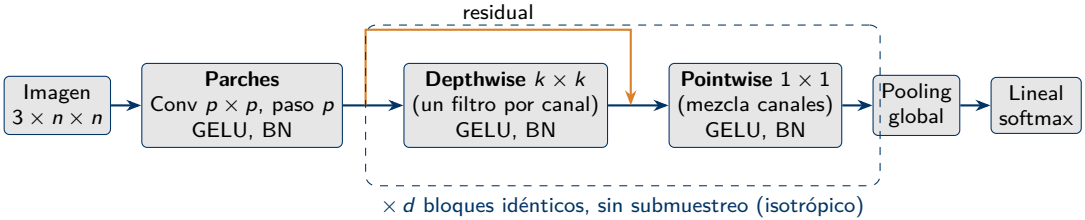
Pregunta del artículo

¿El ViT rinde por el *transformer* o, al menos en parte, porque usa **parches** como representación?

Por qué importa: es una pregunta sobre el *sesgo inductivo*. Si los parches explican la mejora, se conserva la eficiencia de datos de la convolución sin el costo de la atención: clave con pocos datos, como en imágenes médicas.

Motivación: el modelo *más simple* que conserve lo que comparten ViT y MLP-Mixer, con la atención reemplazada por la operación menos expresiva: la convolución.

El ConvMixer: parches + mezcla espacial + mezcla de canales



$$z_0 = \mathrm{BN}(\sigma\{\mathrm{Conv}_{c\to h}(X,\ \text{stride}{=}p,\ \text{kernel}{=}p)\})$$
$$z'_l = \mathrm{BN}(\sigma\{\mathrm{ConvDepthwise}(z_{l-1})\}) + z_{l-1}$$
$$z_{l+1} = \mathrm{BN}(\sigma\{\mathrm{ConvPointwise}(z'_l)\})$$

ConvMixer-h/d: ancho h, profundidad d, parche p, núcleo k.
$\#\theta = h[d(k^2 + h + 6) + c\,p^2 + n + 3] + n$ (ec. 4)
Núcleos **grandes** ($k = 7$–9) para mezclar posiciones lejanas, como la atención.

Nota. Elaboración propia a partir de Trockman y Kolter (2023, Fig. 2 y ecs. 1–4). Aprendizaje supervisado; entropía cruzada; AdamW.

Cómo encaja con la Unidad 4, y qué no vimos en el curso

Lo que es de la U4

- Conv 2D discreta: el *embebido de parches* es una conv con paso = núcleo.
- *Depthwise* + *pointwise* = conv separable (MobileNet).
- SGD (AdamW), BatchNorm y conexión residual.
- Campo receptivo: $p\,[1 + d(k-1)]$ píxeles.

Comparados que no se ven en el curso

ViT/DeiT (Touvron et al., 2021): *transformer* sobre parches; cada parche pondera a todos los demás ($\mathrm{softmax}(QK^\top/\sqrt{d})V$).

MLP-Mixer (Tolstikhin et al., 2021) **/ResMLP**: una MLP mezcla parches y otra mezcla canales.

Swin (Liu et al., 2021): transformer jerárquico con ventanas locales.

Clave

El ConvMixer es un MLP-Mixer con convoluciones en lugar de MLP. Si rinde parecido, lo que importa es lo que comparten: parches y diseño isotrópico.

Metodología de los autores

Datos: ImageNet-1k (1,28 M imágenes, 1 000 clases, 224^2) y CIFAR-10 (50 k + 10 k, 32^2, 10 clases). Sin pre-entrenamiento.

Preprocesamiento: normalización y aumentación intensa con `timm`: RandAugment, mixup, CutMix, *random erasing*, escalado aleatorio.

Características: no hay selección explícita; los parches *aprenden* los filtros.

Partición: la estándar; informan sobre la validación de ImageNet y el test de CIFAR-10.

Entrenamiento: AdamW, tasa de aprendizaje triangular, recorte de la norma del gradiente; 90–300 épocas (unos 9 días en 10 GPU) y 200 en CIFAR-10.

Hiperparámetros: "*absolutamente ninguna*" búsqueda en ImageNet; en CIFAR-10, barrido manual de h, d, p, k y *weight decay*.

Métricas: exactitud top-1, parámetros, imágenes/s.

Ojo (U1)

En CIFAR-10 no reportan un conjunto de validación: los hiperparámetros se eligieron mirando el test.

Resultados del artículo en ImageNet-1k

Modelo	Parche	Parám. (M)	Img/s	Épocas	Top-1 (%)
ConvMixer-1536/20	7	51,6	89	150	**81,37**
ConvMixer-768/32	7	21,1	203	300	**80,16**
ResNet-152 (misma receta)	–	60,2	872	150	79,64
DeiT-B	16	86	703	300	81,8
ResMLP-B24/8	8	129	140	400	81,0
ViT-B/16	16	86	704	300	77,9
Mixer-B/16	16	59	816	300	76,44

+ Con igual presupuesto de parámetros supera a ResNet, ViT, Mixer y ResMLP y queda cerca de DeiT. El 768/32 iguala a la ResNet-152 con un tercio de sus parámetros.

– Es entre 4 y 10 veces más lento. Swin lo supera por 1,2 a 1,6 puntos. Sólo las ResNet se entrenaron con la misma receta; el resto de los números se copia de otros artículos.

Nota. Adaptado de Trockman y Kolter (2023, Tablas 1 y 2). Img/s en GPU RTX8000, lote 64, fp16.

Ablaciones en CIFAR-10 (Apéndice B): núcleos grandes y parches chicos

ConvMixer-256/8, 200 épocas:

- $k = 3 \to 9$: 93,61 $\to$ 95,88 % ($p = 1$).
- $p = 1 \to 2 \to 4 \to 8$: 95,88 $\to$ 95,00 $\to$ 92,61 $\to$ 85,57 %.
- Más de 96 % con menos de 0,9 M parámetros.
- GELU$\to$ReLU: $-0{,}4$; BN$\to$LN: $-1{,}4$.

Nota. Datos de Trockman y Kolter (2023, Tablas 3 y 4).

La aumentación pesa más que la arquitectura:

- sin escalado aleatorio: 86,24 %;
- sin recorte de gradiente: 86,33 %;
- sin RandAugment: 92,86 %.

Clave

Quitar una aumentación cuesta hasta 9,6 puntos; cambiar el núcleo, 2,3. La receta de entrenamiento es parte del resultado tanto como la arquitectura.

Conclusiones y limitaciones del artículo

Conclusiones de los autores

- Lo isotrópico con parches es en sí una plantilla potente: los parches hacen todo el submuestreo de una vez y agrandan el campo receptivo.
- "*Tokenizar*" la entrada es otra idea que la visión tomó del lenguaje.

Trabajo futuro: parches más grandes con más profundidad, optimizar la conv depthwise, segmentación, pre-entrenamiento masivo.

Limitaciones (mi lectura)

1. No hay un diseño factorial que aísle el efecto de los parches (cf. Sandler et al., 2019).
2. Recetas distintas entre los modelos comparados.
3. Una sola corrida por modelo, sin barras de error.
4. Hiperparámetros de CIFAR-10 elegidos mirando el test.
5. Mucho cómputo (unos 90 días-GPU) y poca velocidad de inferencia.

Réplica: el experimento del Apéndice B, a escala de CPU

Igual que el artículo

- **Datos**: CIFAR-10 (Krizhevsky, 2009): `cs.toronto.edu/~kriz/cifar-10.html`.
- Arquitectura: su Fig. 7 literal; los parámetros coinciden con la ec. (4).
- AdamW, tasa triangular, recorte de gradiente, escalado aleatorio, *weight decay* 10^{-2}.
- Comparación con una **ResNet con la misma receta**, y ablaciones de núcleo y de parche.

Distinto, por cómputo (CPU, sin GPU)

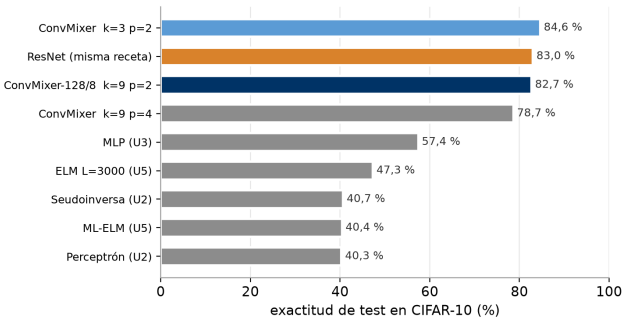
- 10 épocas en lugar de 200.
- ConvMixer-128/8, $p = 2$ (no 256/8, $p = 1$: 3,5 h *por época*).
- Sin RandAugment, mixup ni CutMix.

Más estricto que el artículo (U1)

45 000 / 5 000 / 10 000 (entr./valid./test). Test mirado **una sola vez**.

Nota. Código, semillas e hiperparámetros: `src/`, `config.json` y `notebooks/replica_convmixer.ipynb`.

Resultado: las CNN quedan muy arriba; ConvMixer y ResNet empatan



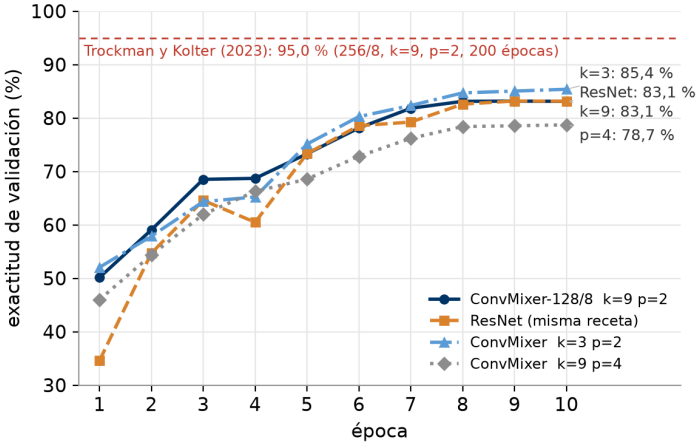
ConvMixer-128/8: 82,7 % con 223 370 parámetros.
ResNet, misma receta: 83,0 % con 308 650.

El MLP tiene 16 veces más parámetros y queda 25 puntos abajo: lo que importa es el **sesgo inductivo** (localidad y pesos compartidos), no la cantidad de parámetros.

Lineales $\approx$ 40 %; la ELM mejora con más neuronas, pero sus rasgos aleatorios no ven la geometría de la imagen.

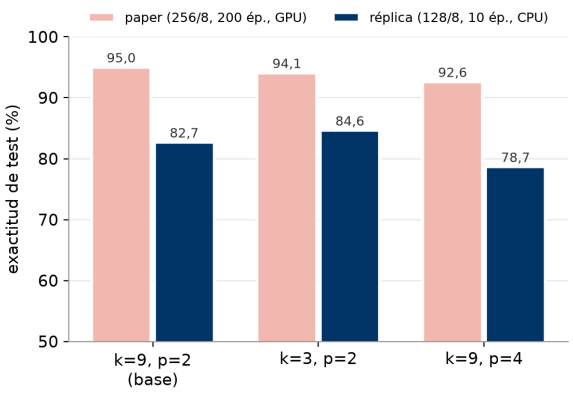
Nota. Exactitud de test (10 000 imágenes). Modelos no convolucionales: promedio de 3 semillas. Elaboración propia.

Curvas de validación: con 10 épocas todos siguen aprendiendo



Nota. La validación se estabiliza al final porque la tasa de aprendizaje llega a cero, no porque el modelo haya terminado de aprender. Elaboración propia.

Ablaciones frente a la Tabla 4: qué se reprodujo y qué no



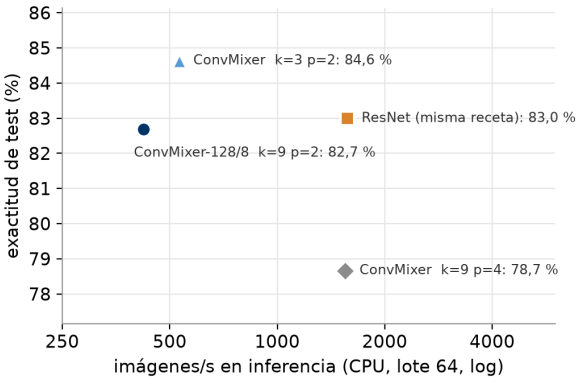
Parche: se reproduce. $p = 4$ pierde 4,0 puntos (paper: 2,4). Menos resolución interna, menos detalle.

Núcleo: no se reproduce. $k = 3$ *gana* por 2,0 puntos. Con pocas épocas converge antes el modelo más simple; los autores mismos dicen que el efecto de k grande podría depender de entrenar más. Una sola semilla: indicativo, no concluyente.

ConvMixer vs. ResNet: 0,3 puntos, dentro del ruido. La ventaja del paper aparece en ImageNet con modelos 100 veces más grandes.

Nota. Réplica: ConvMixer-128/8, 10 épocas, CPU. Artículo: ConvMixer-256/8, 200 épocas, GPU (Trockman y Kolter, 2023, Tabla 4).

El costo sí se reproduce: el ConvMixer es lento



Nota. Lote de 64 imágenes, misma CPU de 8 núcleos, eje logarítmico. Elaboración propia.

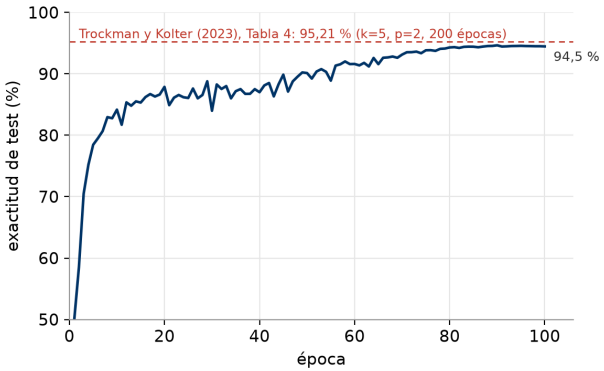
3,7×

más rápida la ResNet en inferencia

(1572 frente a 422 imágenes/s en CPU)

Entrenamiento: 110 min el ConvMixer, 32 min la ResNet. La convolución depthwise de núcleo grande sobre una resolución interna alta está poco optimizada, igual que lo admiten los autores (Tablas 1 y 2).

¿Y si le doy el cómputo del paper? Réplica en GPU (Google Colab)



94,5 %

ConvMixer-256/8, $k = 5$, $p = 2$, receta oficial de los autores

(paper: 95,21 %)

GPU T4 gratuita, unos 56 s por época

con RandAugment, ColorJitter y *random erasing*. La brecha de la réplica en CPU se explica por la escala, no por la implementación.

Nota. Receta de `github.com/locuslab/convmixer-cifar10` (Trockman y Kolter, 2023). Se informa la última época, al cabo de las 100 épocas.

CPU frente a GPU: el cómputo es la variable que más pesa

	Mi computadora (CPU)	**Google Colab (GPU T4)**	**Artículo (GPU)**
Modelo	ConvMixer-128/8, k=9, p=2	ConvMixer-256/8, k=5, p=2	256/8, k=9 o 5, p=2
Parámetros	223 k	594 k	709 k / 594 k
Aumentación	escalado, recorte, espejo	+ RandAugment, ColorJitter, borrado	`timm` + mixup, CutMix
Épocas	10	100	200
Tiempo por época	11 min	56 s	–
Tiempo total	110 min	93 min	–
Exactitud de test	**82,7 %**	**94,5 %**	**95,0 / 95,21 %**

Lectura: en un tiempo parecido (entre 1,5 y 2 horas), la GPU entrena un modelo 2,7 veces más grande, con muchas más épocas y con aumentación completa: **12 puntos más** con el mismo código.

Para el debate: ¿cuánto de "el mejor modelo" es en realidad "el que más cómputo recibió"? Es la misma crítica que le hago al artículo cuando compara recetas distintas, y conecta con la ética: sin GPU sólo se verifica una versión reducida.

Nota. Una corrida por configuración. Colab: se informa la última época, al cabo de las 100 épocas. Valores del artículo: Trockman y Kolter (2023, Tabla 4). Elaboración propia.

Reflexión crítica

¿Por qué este artículo? Es bibliografía que trabajé en el ITBA; tiene una pregunta científica clara y es replicable en hardware modesto; y se conecta con mi tesis (IA para la evaluación de la inestabilidad espinal), donde elegir entre CNN y transformers con pocos datos es una decisión real.

Otros dominios: segmentación médica (la resolución interna alta conserva detalle anatómico), microscopía, teledetección, señales 1D (ECG) y espectrogramas.

Conocimientos previos: U3 (retropropagación, optimización), U4 (convolución, SGD), U1 (validación); depthwise, BatchNorm, residuales, AdamW, aumentación, atención y ViT.

Ética

- **Cómputo**: unos 90 días-GPU por un punto de ImageNet; sin GPU sólo se verifica una versión reducida, y la investigación se concentra en quien tiene hardware.
- **Reproducibilidad**: el código abierto ayuda; una sola corrida y el ajuste sobre el test, no.
- **Sesgos**: ImageNet no representa a todas las poblaciones; en medicina hace falta validación externa local.
- **Doble uso**: la misma tecnología sirve para diagnóstico y para vigilancia.

Conclusiones

Sobre el artículo

- La idea es fuerte y simple: una CNN con parches y diseño isotrópico compite con los transformers.
- La evidencia es parcial: sin diseño factorial, con recetas distintas entre modelos y una sola corrida por configuración.
- Sí muestra algo valioso: la receta de entrenamiento pesa tanto como la arquitectura.

Sobre la réplica

- Con la receta oficial en GPU: 94,5 % (paper: 95,21 %). La implementación es correcta.
- Con poco cómputo: ConvMixer $\approx$ ResNet (82,7 vs. 83,0 %), pero la ResNet es 3,7 veces más rápida.
- Se reproduce el efecto del parche; el del núcleo grande, no con 10 épocas.
- Modelos del curso: lineal $\approx 40 <$ ELM 47 $<$ MLP 57 $\ll$ CNN 83–85 %.

Clave

Replicar no es copiar un número: es seguir el procedimiento, medir lo mismo y poder explicar cada diferencia. Aquí cada brecha tiene una causa concreta: épocas, tamaño, aumentación o la semilla.

Reproducibilidad y referencias

Repositorio: `src/` (modelos, entrenamiento, experimentos, figuras), `config.json` (todas las semillas e hiperparámetros), `results/` (JSON y pesos), `notebooks/replica_convmixer.ipynb` (corre en Colab con GPU, o en modo rápido en CPU).

Todo número de estas diapositivas se genera desde `results/*.json`.

Asistente de IA: el código y el texto se prepararon con Claude Code. La verificación fue mía: la ec. (4) contra el conteo de PyTorch y el test evaluado una sola vez.

Referencias (APA 7)

Dosovitskiy, A. et al. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*. https://doi.org/10.48550/arXiv.2010.11929

Freund, Y. y Schapire, R. E. (1999). Large margin classification using the perceptron algorithm. *Machine Learning*, *37*(3), 277–296.

He, K., Zhang, X., Ren, S. y Sun, J. (2016). Deep residual learning for image recognition. *CVPR*, 770–778. https://doi.org/10.1109/CVPR.2016.90

Huang, G.-B., Zhu, Q.-Y. y Siew, C.-K. (2006). Extreme learning machine: Theory and applications. *Neurocomputing*, *70*(1–3), 489–501. https://doi.org/10.1016/j.neucom.2005.12.126

Kasun, L. L. C., Zhou, H., Huang, G.-B. y Vong, C. M. (2013). Representational learning with ELMs for big data. *IEEE Intelligent Systems*, *28*(6), 31–34.

Krizhevsky, A. (2009). *Learning multiple layers of features from tiny images* [Informe técnico]. University of Toronto. https://www.cs.toronto.edu/~kriz/cifar-10.html

Liu, Z. et al. (2021). Swin Transformer. *ICCV*, 10012–10022. https://doi.org/10.48550/arXiv.2103.14030

Sandler, M., Baccash, J., Zhmoginov, A. y Howard, A. (2019). Non-discriminative data or weak model? *ICCV Workshops*.

Tolstikhin, I. et al. (2021). MLP-Mixer: An all-MLP architecture for vision. *NeurIPS*, *34*, 24261–24272.

Touvron, H. et al. (2021). Training data-efficient image transformers & distillation through attention. *ICML*, 10347–10357.

Trockman, A. y Kolter, J. Z. (2023). Patches are all you need? *Transactions on Machine Learning Research*. https://doi.org/10.48550/arXiv.2201.09792

¡Gracias!

Preguntas y debate



Anexo A: el modelo completo en PyTorch (Fig. 7 del artículo)

```
def ConvMixer(dim, depth, kernel_size=9,
              patch_size=7, n_classes=1000):
  return nn.Sequential(
    nn.Conv2d(3, dim, kernel_size=patch_size,
              stride=patch_size),
    nn.GELU(), nn.BatchNorm2d(dim),
    *[nn.Sequential(
        Residual(nn.Sequential(
          nn.Conv2d(dim, dim, kernel_size,
                    groups=dim, padding="same"),
          nn.GELU(), nn.BatchNorm2d(dim))),
        nn.Conv2d(dim, dim, kernel_size=1),
        nn.GELU(), nn.BatchNorm2d(dim)
      ) for i in range(depth)],
    nn.AdaptiveAvgPool2d((1,1)), nn.Flatten(),
    nn.Linear(dim, n_classes))
```

Verificación de la ec. (4) (PyTorch = fórmula):

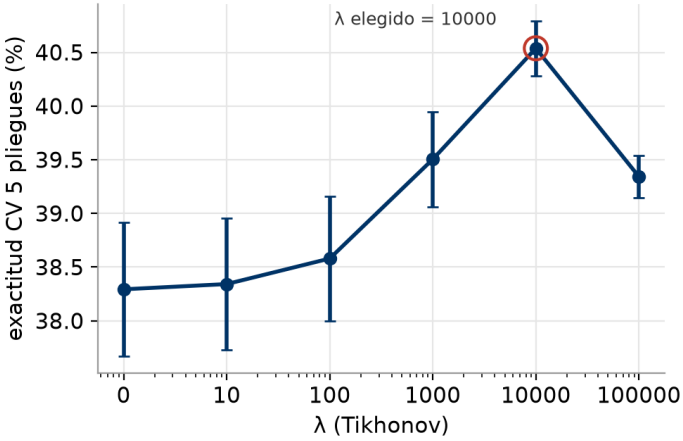
Modelo	Parámetros
256/8, k=9, p=1	706 570
256/8, k=9, p=2	708 874
128/8, k=9, p=2	223 370
128/8, k=3, p=2	149 642

Coinciden con la Tabla 4 del artículo (707 k y 709 k).

Anexo B: los modelos de las otras unidades, en una línea cada uno

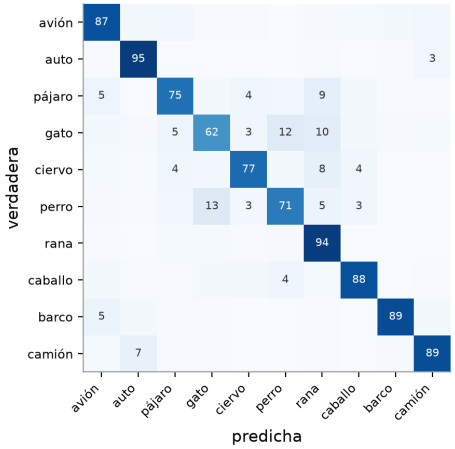
U2 Seudoinversa	$W = V\,\mathrm{diag}\big(\frac{\sigma_i}{\sigma_i^2+\lambda}\big)\,U^\top T$ sobre 3 072 píxeles + sesgo; λ por validación cruzada de 5 pliegues, con una SVD por pliegue.
U2 Perceptrón	Una neurona escalón por clase; si se equivoca, $w_y \mathrel{+}= x$ y $w_{\hat{y}} \mathrel{-}= x$; pesos promediados (Freund y Schapire, 1999).
U3 MLP	3072–1024–512–10, ReLU, *dropout* 0,2, AdamW + programa triangular, 15 épocas.
U5 ELM	$H = \mathrm{ReLU}(XA + b)$ con A, b aleatorios fijos; $\beta = H^+ T$ (Tikhonov); $L \in \{500, \ldots, 3000\}$.
U5 ML-ELM	Dos ELM-autoencoders (1 000 + 1 000, pesos ortogonales) + ELM de 3 000 (Kasun et al., 2013).
U4 ResNet	Tallo 3×3 + 3 bloques residuales (32, 64, 128 canales; resolución $32 \to 16 \to 8$), *pooling* global; misma receta que el ConvMixer.

Anexo C: curva de validación cruzada de la seudoinversa (U1 + U2)



Nota. Exactitud media ± desvío en 5 pliegues de las 45 000 imágenes de entrenamiento. El test se usa una sola vez, con el λ elegido.

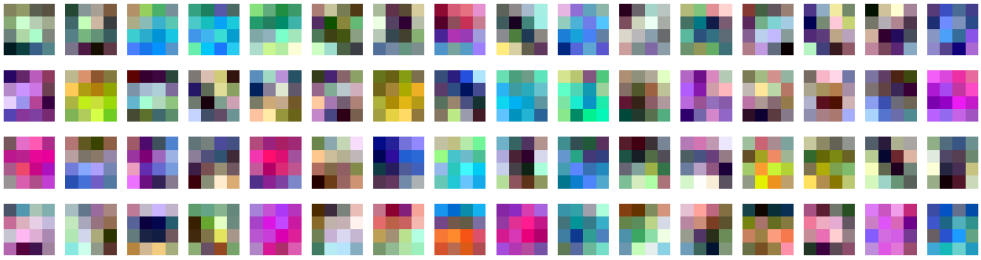
Anexo D: matriz de confusión del ConvMixer-128/8 (test, % por fila)



Nota. Las confusiones se concentran entre clases parecidas: perro con gato (13 %) y gato con perro (12 %); los animales chicos se confunden con rana y el camión con el auto. Gato es la clase más difícil (62 % de acierto).

Elaboración propia.

Anexo E: filtros del embebido de parches (cf. Apéndice C del artículo)



Nota. 64 filtros RGB de 4×4 del ConvMixer con $p = 4$: bordes orientados y manchas de color, como los que describen Trockman y Kolter (2023). Elaboración propia.